

GitHub 上传与团队协作说明

由 Markdown 阅读资料转换生成 · 通用自适应阅读版 · 适合转发和专家审阅

GitHub 上传与团队协作说明

更新时间：2026-05-02

适用项目：杨凌国合外贸综合服务平台 1.0

一、先理解几个概念

名词	通俗理解
Git	你电脑里的“代码修改记录本”，能记录谁改了 什么、什么时候改的
GitHub	放在网上的代码仓库，团队成员都能同步代码
仓库 repository	一个项目的代码库
commit	一次修改记录，比如“修复配置页错误”
push	把你电脑上的修改上传到 GitHub
pull	把 GitHub 上别人改的内容拉到你电脑
branch	分支，适合每个人单独改功能，最后再合并
Pull Request	合并申请，团队看完代码后再合并到主分支

你们现在 3 个人协作，建议一开始就养成这个习惯：

每个人单独建分支修改
改完提交 `commit`
推送到 `GitHub`
通过 `Pull Request` 合并到 `main`

不要 3 个人都直接改 `main`，否则很容易互相覆盖。

二、上传前不要传哪些东西

项目根目录已增加 `.gitignore`，默认不会上传这些内容：

- `frontend/node_modules/`：前端依赖，太大，别人用 `npm ci` 安装即可。
- `frontend/dist/`：前端构建产物，可重新构建。
- `backend/target/`：后端 JAR 构建产物，可重新打包。
- `deploy/dist/`、`deploy/*.jar`：部署构建产物，可后续发布版本时单独处理。
- `uploads/`：业务上传文件，可能包含合同、发票、报关资料，不应上传。
- `logs/`、`*.log`：运行日志，不应上传。
- `.env`、本地配置文件、压缩包：避免泄露密码或产生巨大仓库。

特别提醒：`GitHub` 私有仓库也不要上传真实合同、报关单、发票、客户资料、身份证、营业执照、银行流水、真实密码。

三、推荐上传方式

你有两种方式：

1. GitHub Desktop：适合第一次使用，界面化。
2. 命令行：更像正式开发流程，建议慢慢学会。

我建议你这次用“网页建仓库 + 命令行上传”，因为以后团队协作迟早要用命令行。

四、在 GitHub 网站创建私人仓库

1. 打开 GitHub。
2. 登录你的账号。
3. 点击右上角 **+**。
4. 选择 **New repository**。
5. Repository name 建议填写：

```
yangling-guohetradetradeplatform
```

1. Description 可填写：

```
杨凌国合外贸综合服务平台 1.0
```

1. 选择 **Private**，一定要选私人仓库。
2. 不要勾选 **Add a README file**。
3. 不要勾选 **.gitignore**。

4. 不要选择 License。

5. 点击 `Create repository`。

创建完成后，GitHub 会给你一个仓库地址，类似：

```
https://github.com/你的用户名/yangling-guohe-trade-platform.git
```

后面命令里要用这个地址。

五、第一次本地初始化并上传

进入项目目录：

```
cd /Users/wangbin/Documents/Codex/review_src2/autoWorkPlatform-v1.0.0
```

初始化 Git：

```
git init
```

设置主分支名称：

```
git branch -M main
```

查看将要上传的文件：

```
git status
```

添加文件：

```
git add .
```

提交第一版：

```
git commit -m "初始化外贸综合服务平台1.0修复版"
```

绑定 GitHub 仓库地址，把下面地址换成你自己 GitHub 创建后的地址：

```
git remote add origin https://github.com/你的用户名/yangling-guohe-trade-platform.git
```

上传：

```
git push -u origin main
```

如果提示登录 GitHub，按提示登录即可。

六、以后每天怎么协作

每天开始工作前先拉最新代码：

```
git pull
```

创建自己的分支，例如你要改服务企业台账：

```
git checkout -b feature/service-company-ledger
```

修改代码后查看改了什么：

```
git status
```

提交修改：

```
git add .
git commit -m "新增服务企业台账基础页面"
```

上传分支：

```
git push -u origin feature/service-company-ledger
```

然后到 GitHub 网站上创建 Pull Request，请另外两个人看一下，再合并到 `main`。

七、团队 3 人最简单分工建议

成员	建议分支	负责方向
你	<code>feature/platform-planning</code>	需求、认定材料、业务流程、验收
同事 A	<code>feature/frontend-pages</code>	前端页面、菜单、表单、交互
同事 B	<code>feature/backend-api</code>	后端接口、数据库、权限、部署

也可以按功能分：

功能	分支名
服务企业台账	<code>feature/service-company-ledger</code>

功能	分支名
外综服合同模板管理	feature/service-contract-template
认定指标统计	feature/recognition-dashboard
PVE Docker 部署	feature/docker-deploy

八、commit 信息怎么写

好的 commit 信息应该让人一眼看懂。

建议：

- 修复配置管理状态更新错误
- 新增外综服补充服务确认函模板
- 新增服务企业台账字段设计
- 修复合同列表路由空白页
- 补齐进口台账数据库表

不建议：

- 修改
- 111
- 更新一下
- 临时改改

九、第一次上传前建议检查

上传前运行：

```
git status
```

如果看到这些目录，不要上传：

```
frontend/node_modules/  
backend/target/  
frontend/dist/  
deploy/dist/  
uploads/  
logs/
```

如果不确定，把 `git status` 的输出发给我，我帮你判断。

十、以后如何恢复或查看谁改了什么

查看提交记录：

```
git log --oneline
```

查看某个文件改了什么：

```
git diff 文件路径
```

查看当前工作区改了什么：

```
git diff
```

查看每个文件是谁最后改的：

git blame 文件路径

十一、重要安全提醒

1. 仓库必须是 Private 。
2. 不要上传真实客户资料和业务附件。
3. 不要上传数据库备份。
4. 不要上传真实服务器密码。
5. 不要上传真实 JWT 密钥、数据库密码、GitHub Token。
6. 后续生产服务器部署用的密码，放在服务器 `.env` 或生产配置里，不提交到 GitHub。

十二、你们目前最适合的节奏

第一步：把当前 1.0 修复版上传到 GitHub，作为基线版本。

第二步：三个人都从 GitHub 拉代码。

第三步：先做 1.1 版本，不要一开始改太大。

1. 服务企业台账。
2. 外综服服务事项记录。
3. 认定指标统计。
4. Docker/PVE 测试部署。
5. 操作日志和权限细化。

第四步：每周固定合并一次，形成一个小版本。

建议版本节奏：

v1.0.0 当前修复版
v1.1.0 服务企业台账和认定基础功能
v1.2.0 PVE Docker 测试部署
v1.5.0 外综服认定材料管理
v2.0.0 省级外综服平台增强版

十三、我建议你下一步做什么

你先在 GitHub 上创建私人仓库，拿到仓库地址。

然后把地址发给我，我可以帮你在本机完成：

```
git init
git add
git commit
git remote add
git push
```

上传前我会再帮你检查一遍，确认没有把 `node_modules`、构建产物、上传文件、日志、压缩包传上去。

本文档为内部研究和项目推进资料，请结合正式政策文件、合同文本和业务凭证审慎使用。